

RI402

Dizajn kompajlera

dr.sc. Edin Pjanić

Pregled predavanja

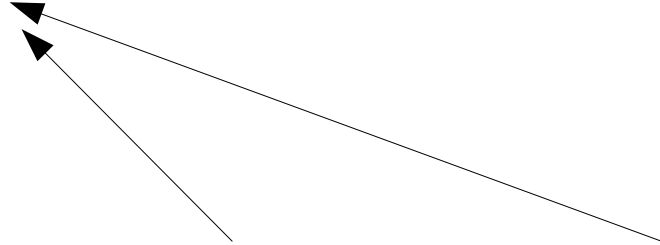
- Implementacija leksičkog analizatora (leksera)
- RegEx -> NFA -> DFA

Input buffer:

```
if( a <= b ) c = 1;
```

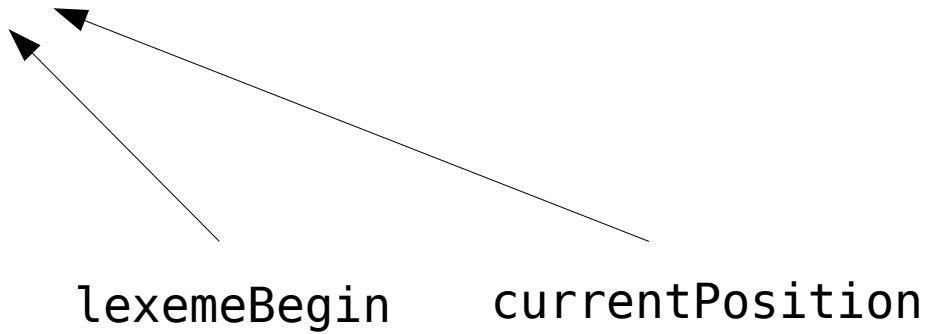
lexemeBegin

currentPosition

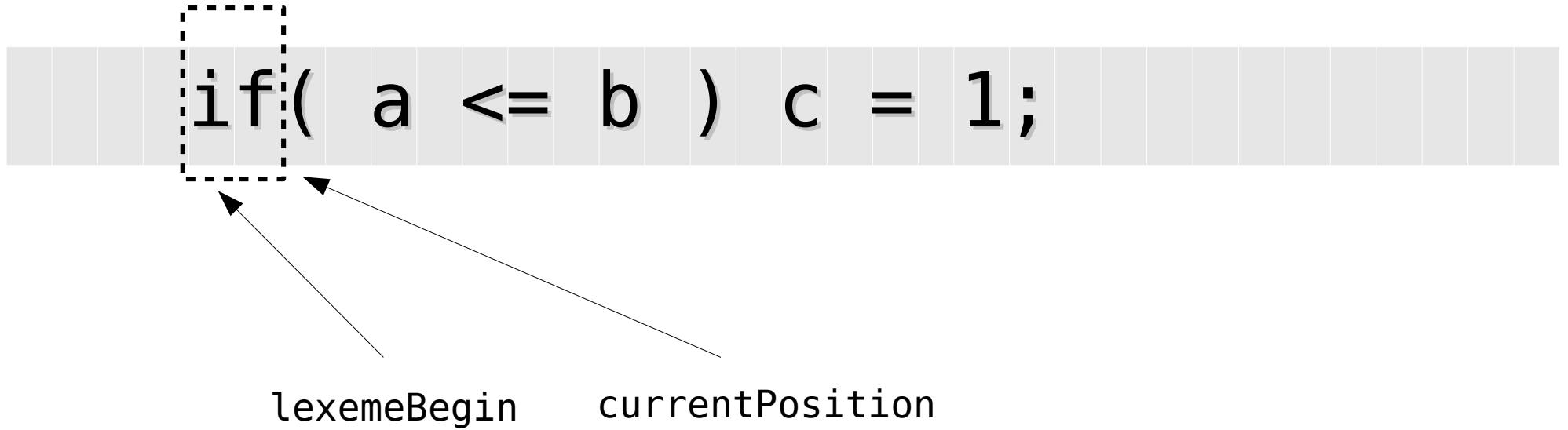


Input buffer:

```
if( a <= b ) c = 1;
```



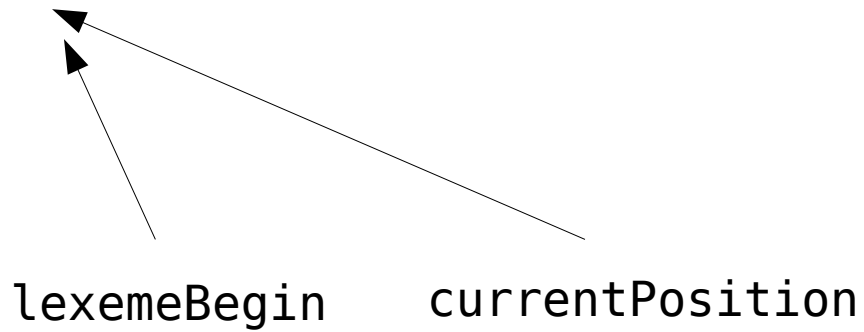
Input buffer:



Token: <IF>

Input buffer:

```
if( a <= b ) c = 1;
```



Token: <IF>

Input buffer:

The diagram shows a horizontal input buffer represented as a grid of 30 cells. The code 'if(a <= b) c = 1;' is stored in the buffer, with spaces filling the remaining cells. A dashed rectangular box highlights the characters 'if(' in the 10th, 11th, and 12th cells. Two arrows point to the bottom-right corner of this box: one from the label 'lexemeBegin' and another from the label 'currentPosition'.

```
if( a <= b ) c = 1;
```

lexemeBegin

currentPosition

Token: <IF>

Token: <OPEN>

Input buffer:

```
if( a <= b ) c = 1;
```

lexemeBegin

currentPosition

Token: <IF>

Token: <OPEN>

Token: <WS, " ">?

Token: <ID, "a">

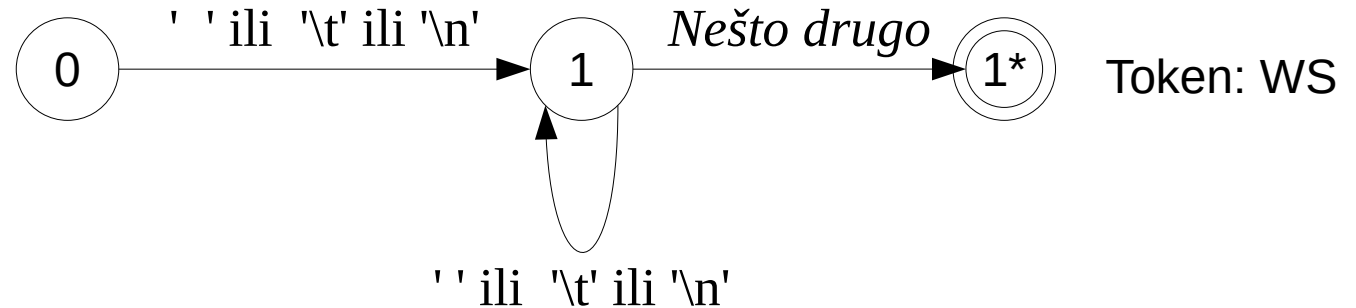
Token: <WS, " ">?

Leksička analiza: implementacija

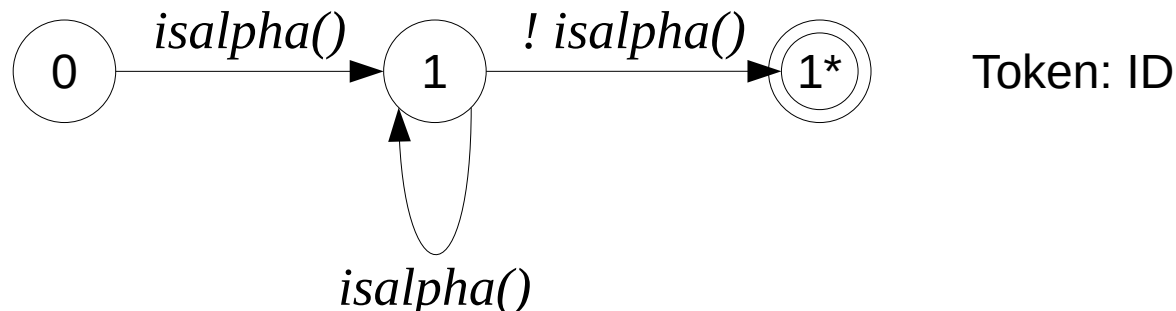
- Potrebno je implementirati leksički analizator za jezik koji se sastoji bar od sljedećih vrsta riječi:
 - Prazna mjesta
 - Identifikatori (sastoje se samo od slova engleskog alfabeta)
 - Brojeva (cijeli brojevi)
 - Operator dodjeljivanja =
- Za svaku vrstu riječi (tokena) formirati regularni izraz i dijagram stanja.
- Napisati programski kod za implementaciju.

Regularni izrazi i dijagrami stanja

- Whitespace regexp: (' ' | '\t' | '\n') +

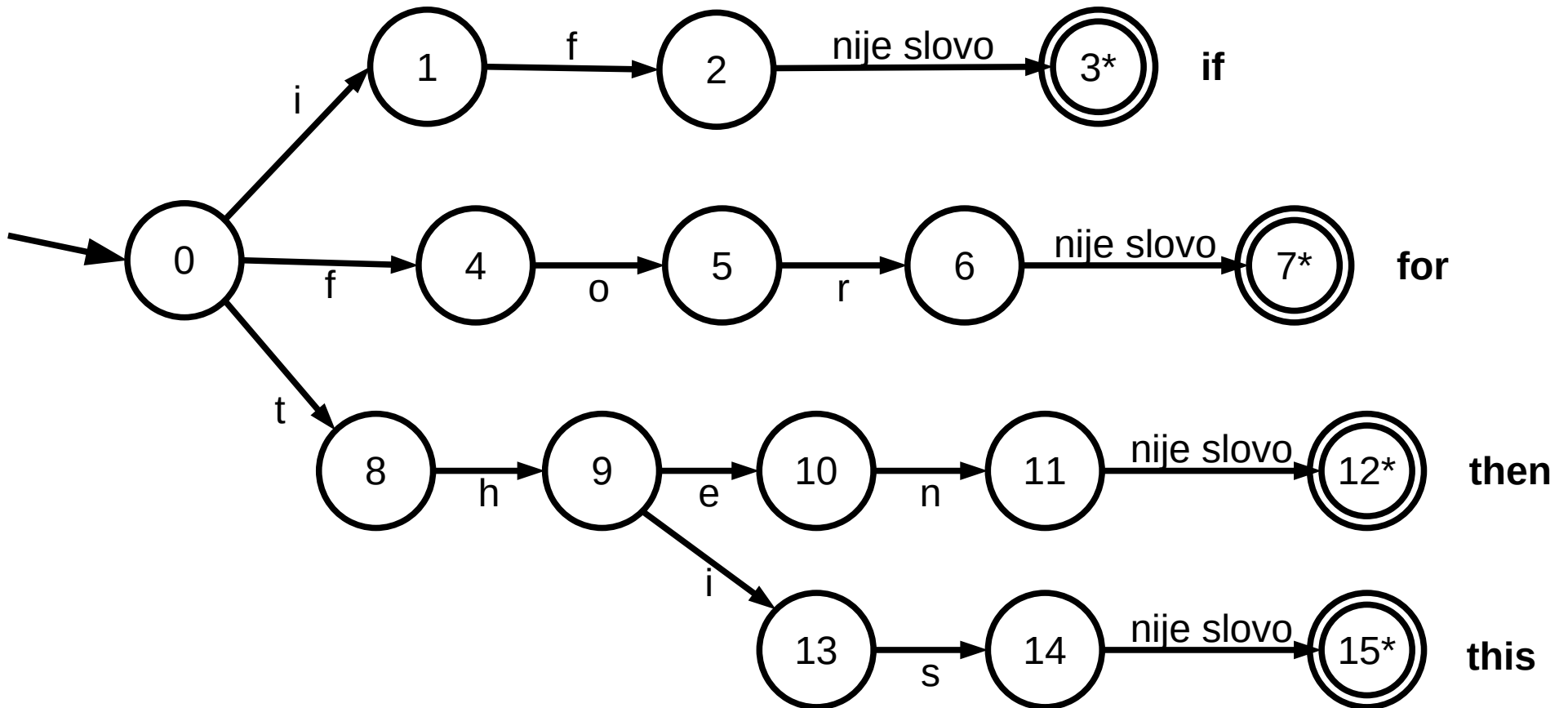


- Neka je regexp $\alpha = [a-zA-Z]$
- Onda je regexp za identifikator:
 - $id = \alpha \alpha^*$



Primjer dijagrama za ključne riječi

Keywords: if | for | then | this



```

Token getKeyword(InputBuffer& b)
{
    b.startScan();
    int state = 0;
    char c;
    do{
        c = b.peekNext();
        switch(state){
            case 0:
                if(c=='i')
                    state = 1;
                else if(c=='f')
                    state = 4;
                else if(c=='t')
                    state = 8;
                else
                    state = 99;
                break;

```

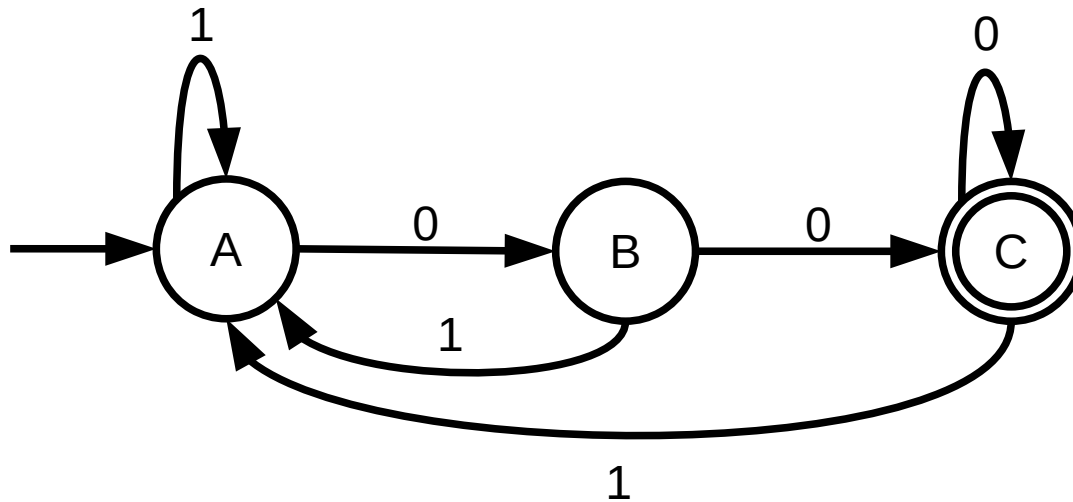
```

            case 1:
                if(c=='f')
                    state = 2;
                else
                    state = 99;
                break;
            case 2:
                if(isalpha(c))
                    state = 99;
                else
                    return Token(IF);
                break;
            case 4:
                if(c=='o')
                    state = 5;
                else
                    state = 99;
                break;
            ...
            case 99:
                return Token(0);
            ...

```

Primjer ideje za implementaciju.

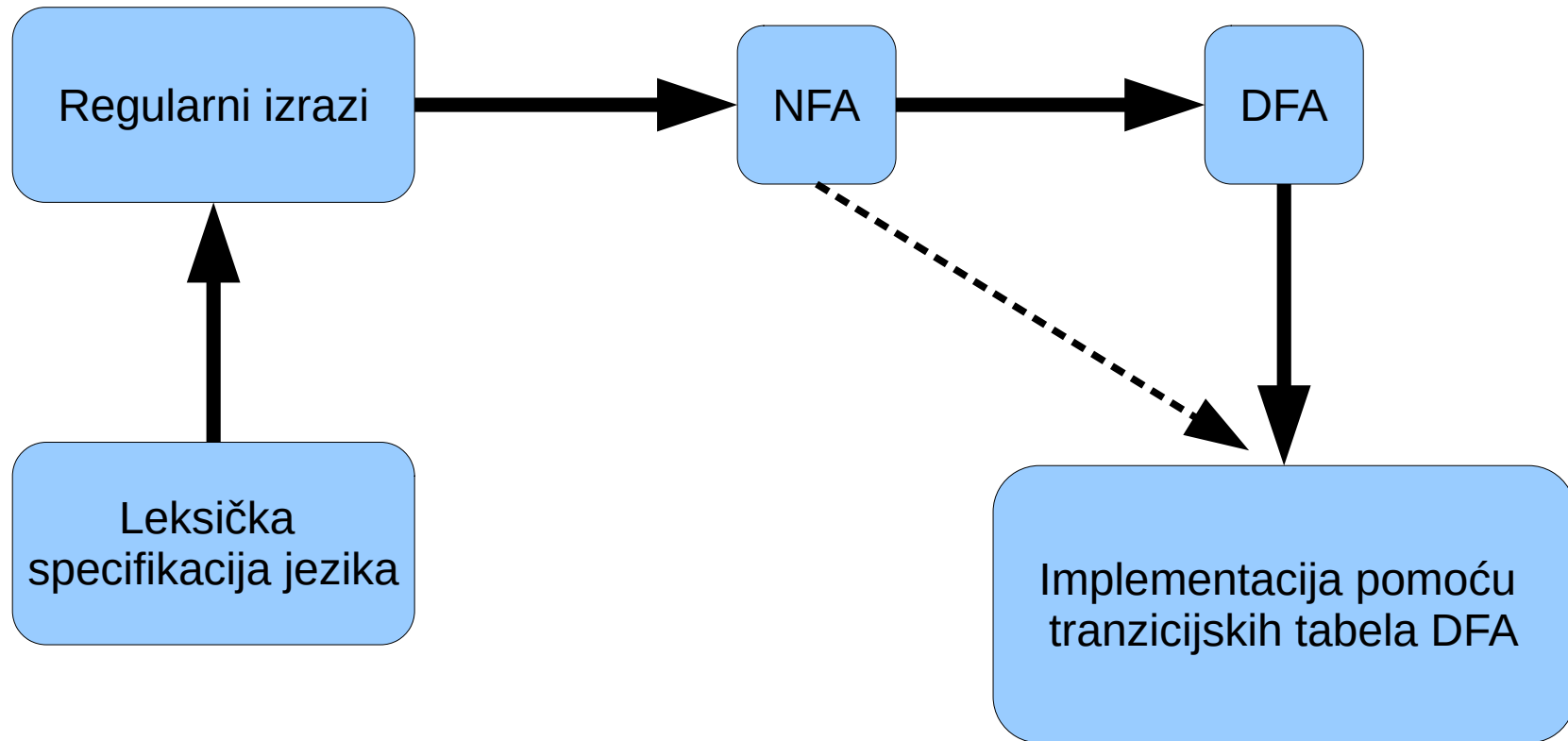
Primjer implementacije 2



Stanje	0	1
A	B	A
B	C	A
C	C	A

i
ulaz: "10010100"

```
while( ulaz[i] ){  
    novoStanje = Tabela[ trenutnoStanje, ulaz[i++] ];  
    trenutnoStanje = novoStanje;  
}
```



Prevođenje regexp u NFA

- Neka je ekvivalentni NFA za regularni izraz M prikazan kao:



NFA i regexp su ekvivalentni ako:
NFA i regexp M prepoznaju iste jezike.

- Ekvivalentni NFA za regexp ε :

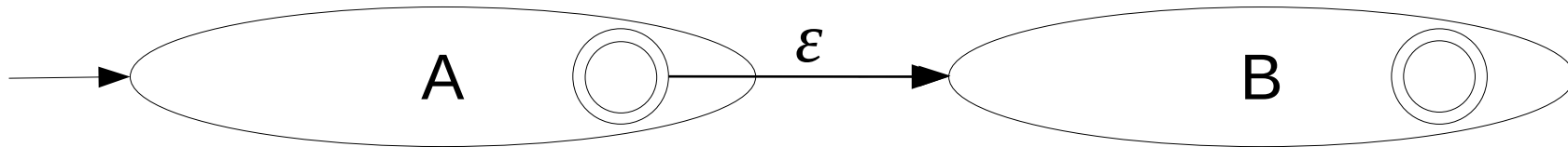


- Ekvivalentni NFA za regexp $'a'$:

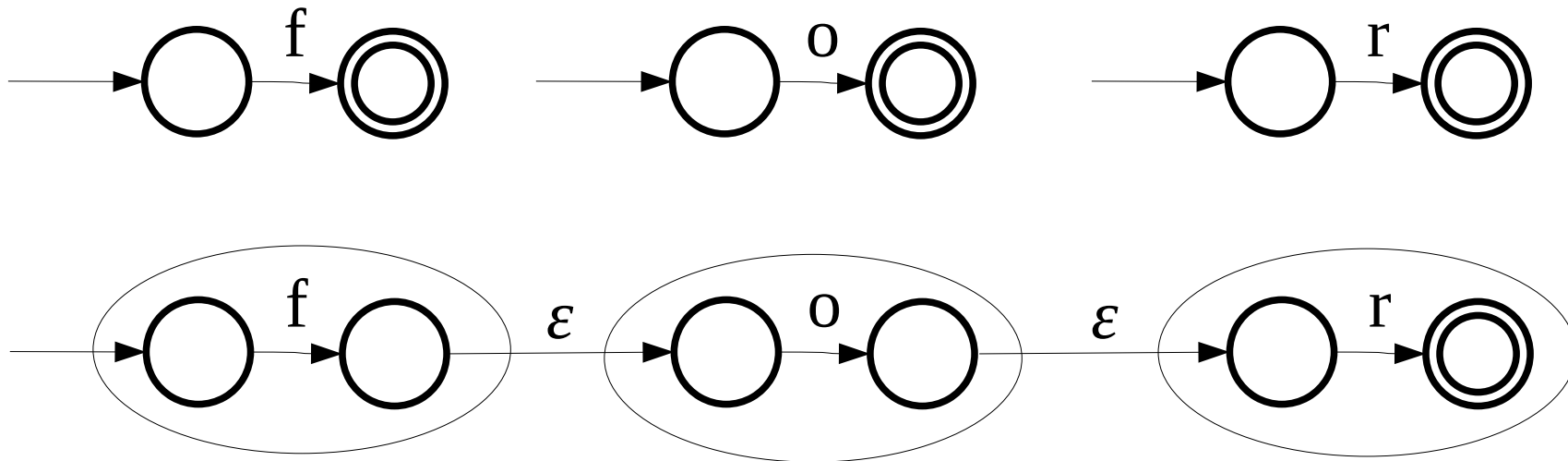


Prevođenje složenijih regexp izraza u NFA

- Ekvivalentni NFA za regexp $A B$:

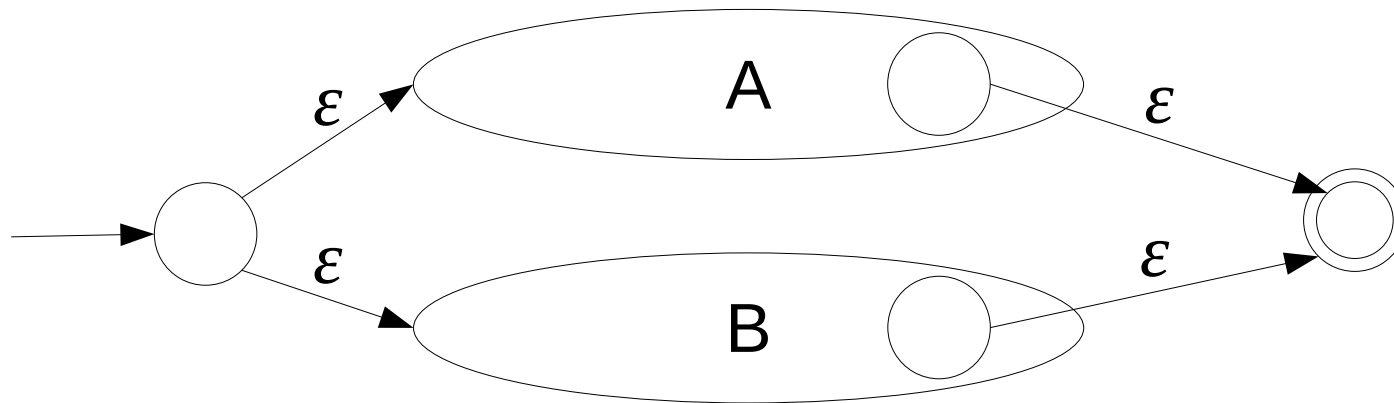


- Primjer: NFA koji prepoznaje regexp 'f' 'o' 'r' (npr. ključna riječ 'for'):



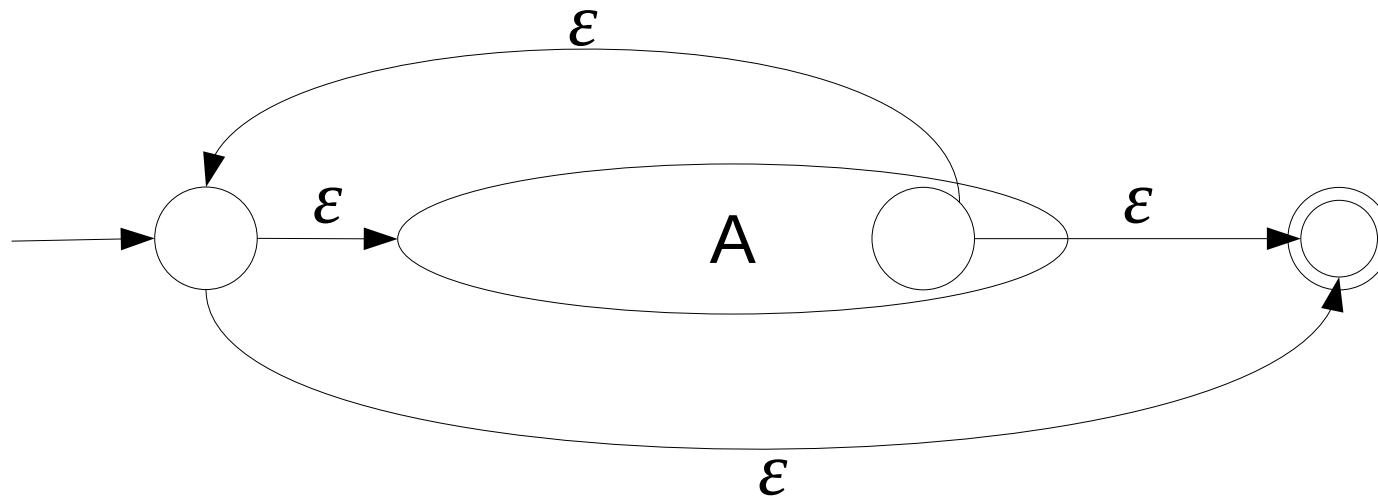
Prevođenje složenijih regexp izraza u NFA

- Ekvivalentni NFA za regexp $A+B$:



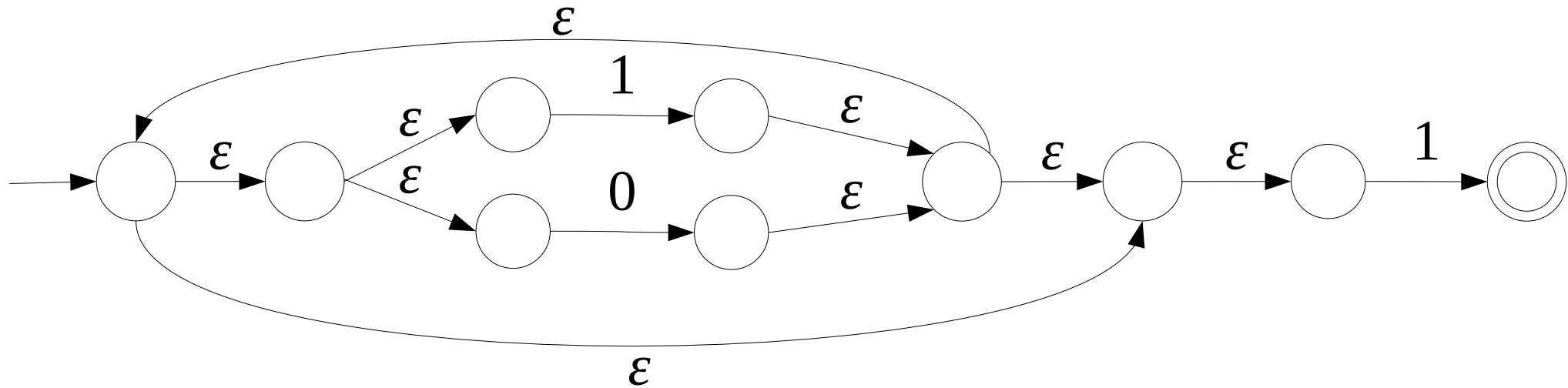
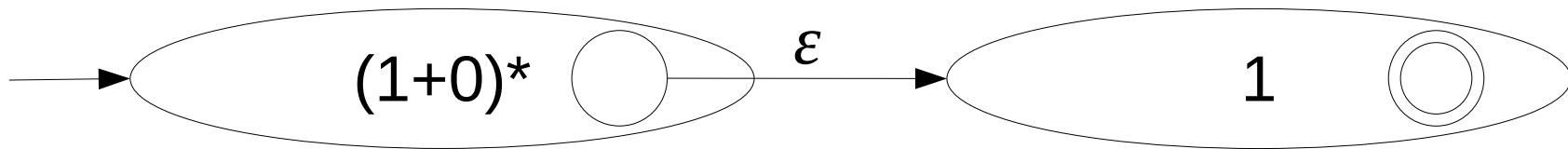
Prevođenje složenijih regexp izraza u NFA

- Za regexp A^* :



Primjer

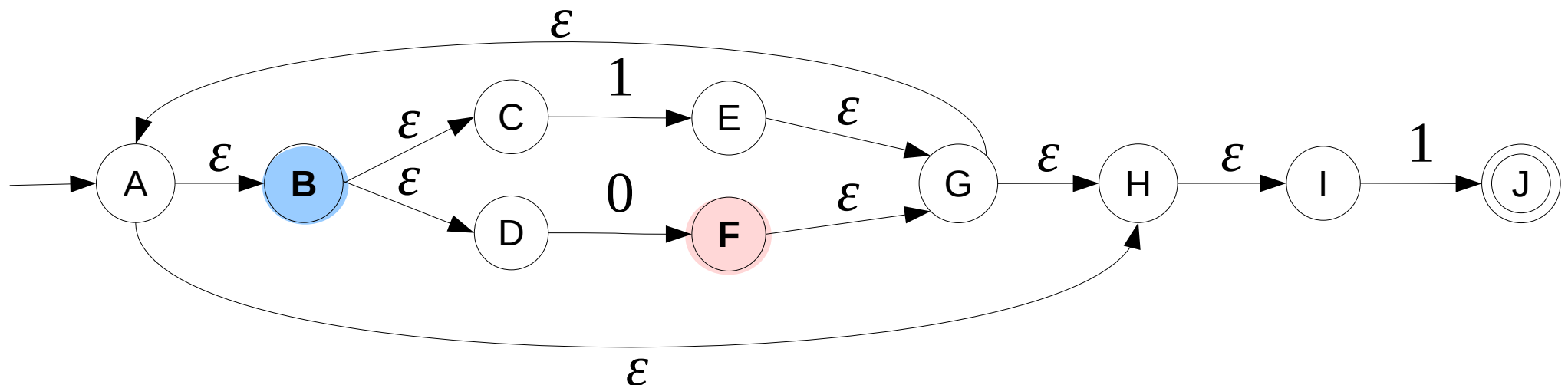
- $(1+0)^*1$:



Prevođenje NFA u DFA

Prevođenje NFA u DFA

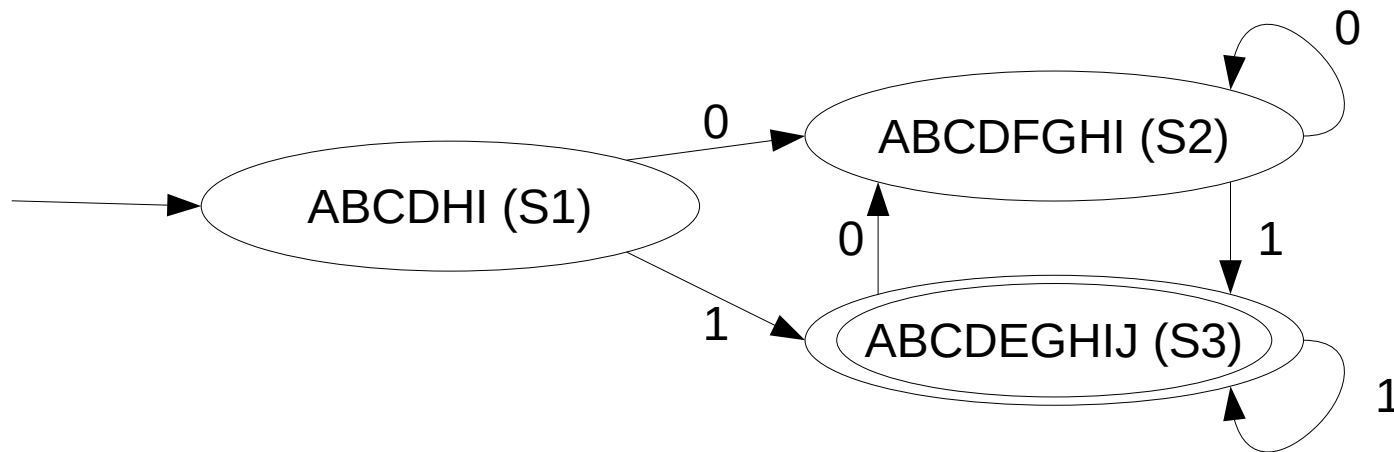
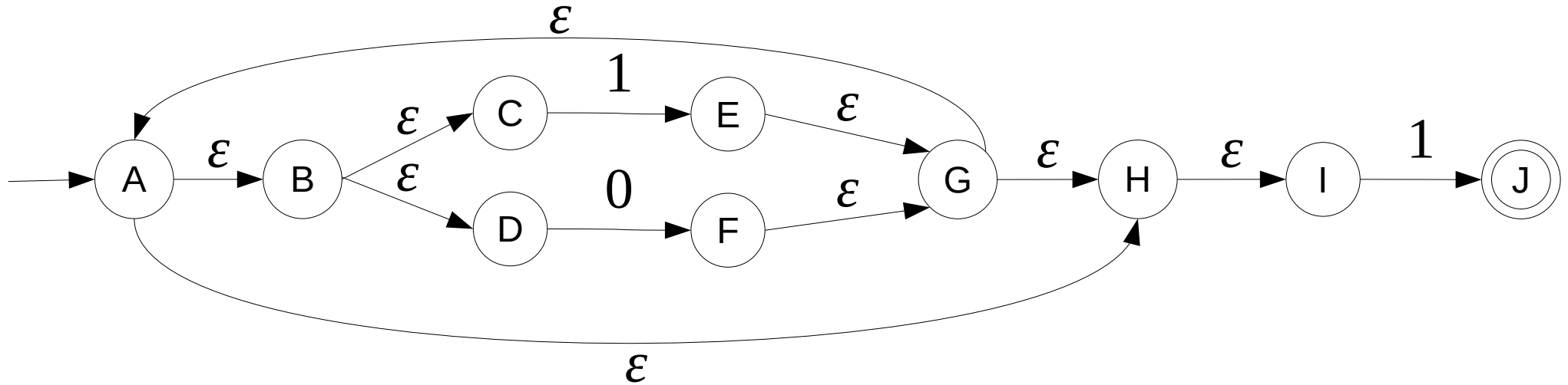
- *Def:*
 - ϵ -zatvorenje (*eng. closure*) za neko stanje: skup svih stanja u koja se može doći iz trenutnog stanja slijedenjem ϵ -prelaza.
Označavamo ϵ -clos.
- $\epsilon\text{-clos}(B) = \{B, C, D\}$
- $\epsilon\text{-clos}(F) = \{A, B, C, D, F, G, H, I\}$



Prevođenje NFA u DFA

- Neka za NFA imamo:
 - *Skup svih stanja: S*
 - *Početno stanje: s*
 - *Skup prihvatajućih stanja: F*
 - *Skup tranzicija za neki ulaz $a(X) = \{y \mid x \in X \text{ i } x \rightarrow^a y\}$*
 - *Za svako stanje, neka je dat skup $\varepsilon\text{-clos}$*
- Ekvivalentni DFA će imati:
 - *Stanja: podskupovi skupa stanja S , osim praznog skupa*
 - *Početno stanje: $\varepsilon\text{-clos}(s)$*
 - *Tranzicije: $X \rightarrow^a Y$ tako da je $Y = \varepsilon\text{-clos}(a(X))$*
 - *Prihvatajuća stanja: $\{X \mid X \cap F \neq \emptyset\}$*

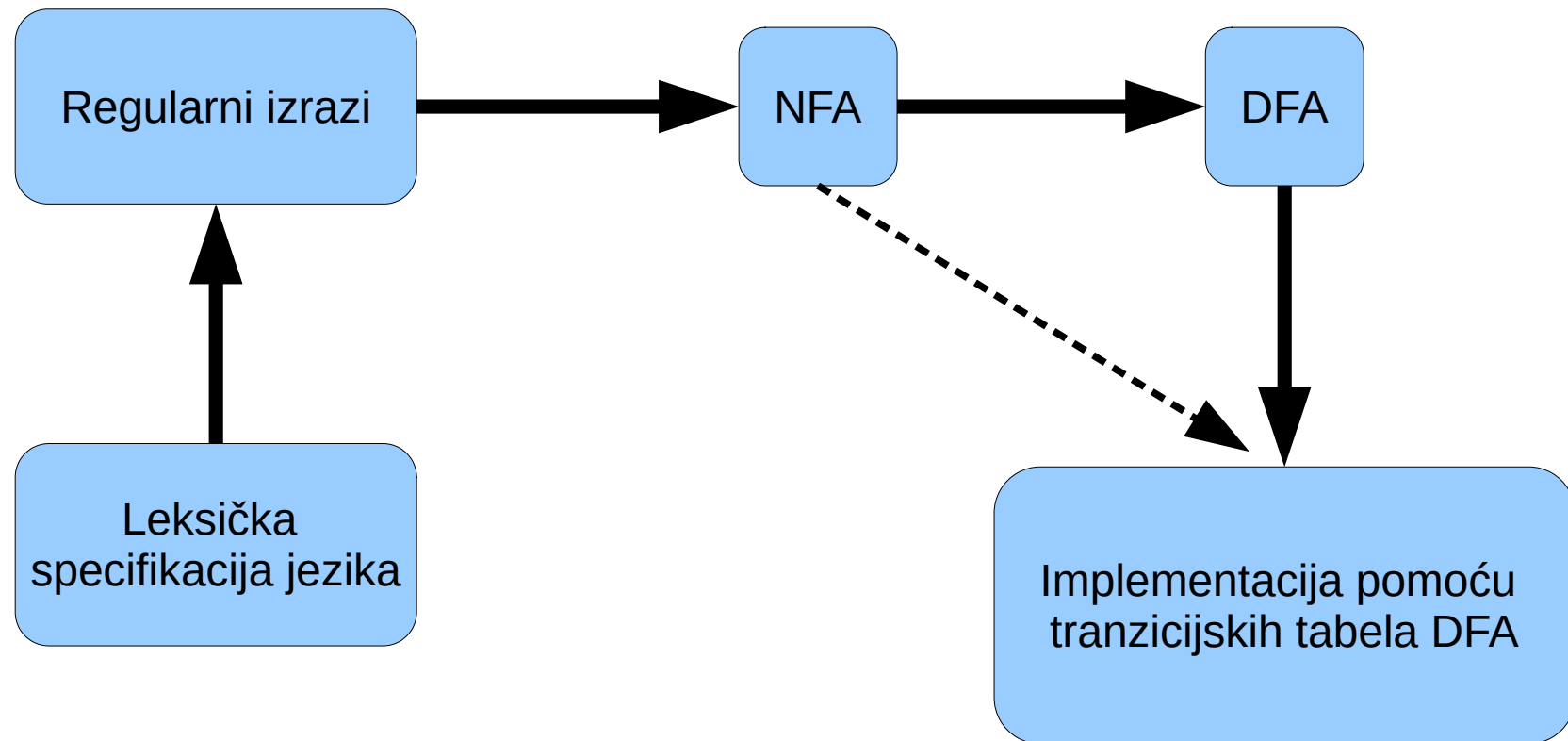
Prevođenje NFA u DFA: Primjer



Stanje	0	1
S1	S2	S3
S2	S2	S3
S3	S2	S3

Prilikom analize i prevođenja, ekvivalentna stanja za DFA crtati u tranzicijsku tabelu

Implementacija leksera

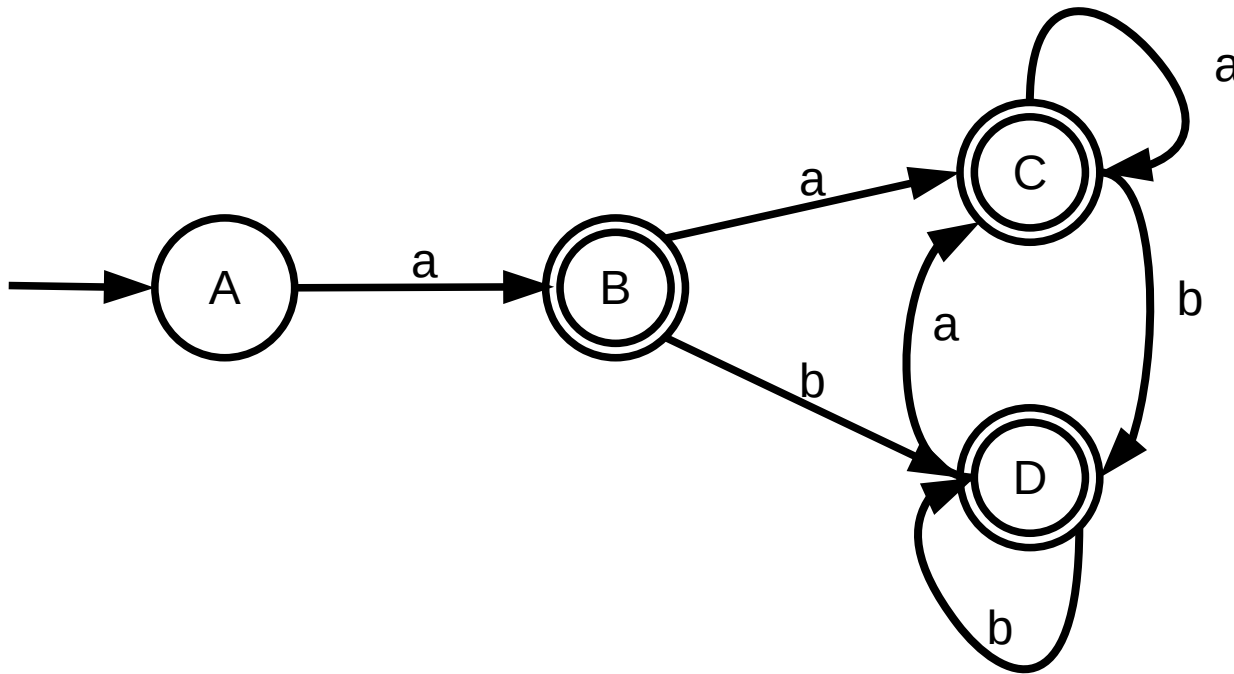


Leksička analiza: implementacija

- Potrebno je implementirati leksički analizator za jezik koji se sastoji bar od sljedećih vrsta riječi:
 - Prazna mjesta
 - Ključne riječi: if, then i for
 - Identifikatori (sastoje se samo od slova engleskog alfabeta)
 - Brojeva (cijeli brojevi)
 - Operator dodjeljivanja =
- Za svaku vrstu riječi (tokena) formirati regularni izraz i dijagram stanja.
- Napisati programski kod za implementaciju.

Analizirajmo DFA koji implementira regularni izraz **$a(a+b)^*$**

$a(a|b)^*$
 $a[ab]^*$



Da li postoji automat sa manjim brojem stanja koji implementira dati regularni izraz?

Regex i automati

- Za jedan regularni izraz može postojati više automata koji ga implementiraju
 - Mogu imati različite oznake stanja
 - Mogu imati različit broj stanja
- Implementacija leksera:
 - Želimo automat sa što manje stanja
 - Nazivi stanja su nam najmanje bitni
- Očito, ako se dva automata mogu transformisati jedan u drugi samo promjenom naziva stanja:
 - Implementiraju isti regularni izraz
- **T:** Za svaki regularni izraz postoji jedinstven automat koji ga implementira sa najmanjim mogućim brojem stanja.
 - Može se konstruisati od bilo kojeg automata koji implementira jezik (grupisanjem ekvivalentnih stanja)

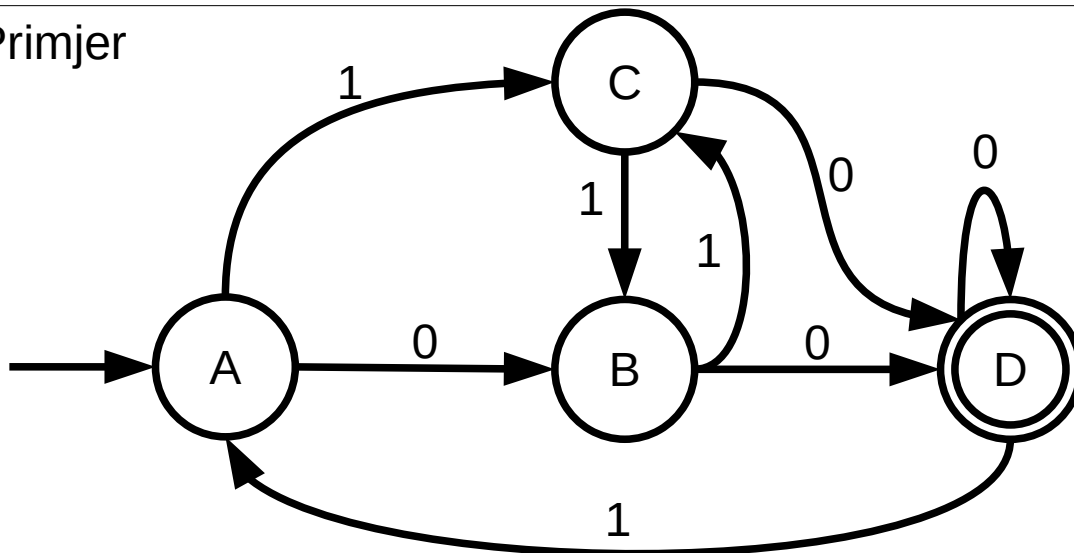
Razlikovanje stanja - različita stanja

- Kažemo da string x razlikuje stanje s od stanja t ako, slijedeći putanju po stringu x počevši od tih stanja, samo jedno od tih stanja vodi do prihvatajućeg stanja.

Posljedica:

Stanje s je različito od stanja t ako postoji neki string koji ih razlikuje. Stringom se smatra i prazan string (ϵ).

Primjer



Stanje D je različito od svih stanja.
Stanje A je različito od stanja B
Stanje B nije različito od stanja C

Minimizacija broja stanja DFA - algoritam

1. Stanja podijeliti u dvije grupe:

- Grupa sa neprihvatajućim stanjima
- Grupa sa prihvatajućim stanjima

2. Za svaku grupu:

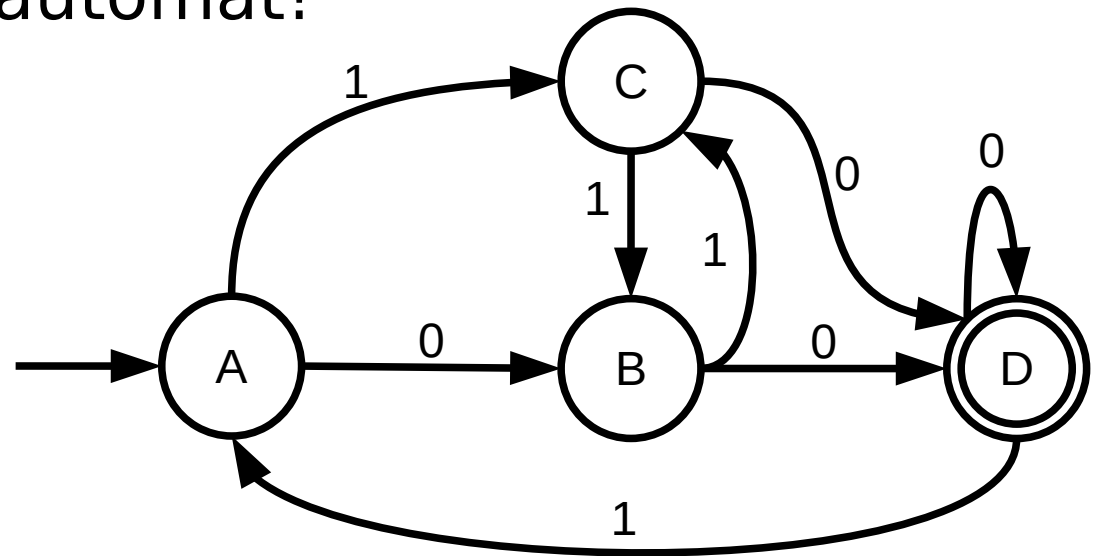
- Podijeliti grupu na podgrupe (P_{ij}) tako da za sva stanja u istoj podgrupi tranzicije na neki simbol idu u neku istu podgrupu
- Najmanja podgrupa je skup od jednog stanja

3. Ponavljati korak 2 sve dok se grupe mogu dijeliti u manje podgrupe

4. Renumerisati stanja tako da svakoj podgrupi odgovara jedno stanje

Primjeri

Primjer 1: Minimizirati automat!



Primjer 2: Dat je regularni izraz **$a(a+b)^*$**

a) Konvertovati regularni izraz u NFA

b) Pronaći ekvivalentni DFA

c) Minimizirati dobijeni DFA